

Ain Shams University

Faculty of Engineering

CSE336 – Software Design Patterns

Non-Profit Organization Management System

Major Assessment Task – Phase 1 Report

Class Diagram, Design Patterns, MVC Architecture, Client Requirements and CRUD Lookup Tables

Team 13		
No.	**Student Name**	**Student ID**
1	Mohamed Magdy Mahmoud	22P0127
2	Mahmoud Nashaat	22P0126
3	Mohamed Ghoneem	22P0124
4	Mohamed Mahdy	22P0089
5	Zakaria Alaa Eisa	22P0128
6	Kareem Younis	22P0136

Presented to: **Dr. Raghda Essam**

GitHub Repository

https://github.com/ZakariaEisa/Non-Profit-Organization-Management-System-

November 2026

Table of Contents

1. Introduction 2
1.1 Project Overview 2
1.2 Objectives 2
1.3 Phase 1 Scope and Deliverables 2
1.4 Team Members and Contributions 3
1.5 GitHub Repository 3
2. System Requirements 4
2.1 Basic Requirements 4
2.2 Client Requirements 4
3. System Architecture (MVC) 6
3.1 Model–View–Controller 6
3.2 Package Structure 6
4. Class Diagram 8
4.1 Core Domain Classes 9
4.2 Main Relationships 11
5. Design Patterns 12
5.1 Singleton Pattern 12
5.2 Factory Pattern 12
5.3 Observer Pattern 13
5.4 Strategy Pattern 14
5.5 Decorator Pattern 15
6. CRUD Lookup Tables 16
7. Database Design 17
8. Implementation and Testing 18
8.1 Implementation 18
8.2 Sample Output 18
8.3 Test Scenarios 18
9. Plan for Phase 2 20
10. Conclusion 21
Deliverables 21

1. Introduction

1.1 Project Overview

Non-profit organizations manage many activities at the same time: they receive donations through different channels, coordinate volunteers, organize fundraisers and workshops, support beneficiaries and keep everyone informed. Doing this with spreadsheets and manual messages is slow and error-prone.

This project designs and builds a **Non-Profit Organization (NPO) Management System** that brings these activities into one application. The system is organized around five modules: Donation Management, Volunteer Management, Event Management, Beneficiary Management and Communication. It is designed with object-oriented design patterns so that new requirements can be added with minimal changes to existing code.

1.2 Objectives

- **Flexibility:** new donation methods, communication channels or receipt features can be added without modifying existing classes.
- **Maintainability:** responsibilities are separated into Model, View and Controller packages, and every class has a single, clear purpose.
- **Reusability:** shared behavior is placed in abstract base classes and interfaces (Person, Donation, LookupEntity, BaseController, IReceipt, INotificationStrategy).

1.3 Phase 1 Scope and Deliverables

Phase 1 covers the complete design of the system and a working implementation of the Phase 1 patterns. Table 1 maps each evaluation item to the deliverable that satisfies it.

Table 1: *Phase 1 evaluation items and where they are delivered*

Evaluation item	Marks	Delivered in
Class diagram	1	Section 4 and NPO_Class_Diagram_UML.pdf
Strategy pattern	0.5	Section 5.4 – notification channels
Observer pattern	0.5	Section 5.3 – donation and event notifications
Decorator pattern	0.5	Section 5.5 – donation receipts
Factory pattern	0.5	Section 5.2 – donation and strategy creation
Singleton pattern	0.5	Section 5.1 – database manager
MVC	0.5	Section 3 – system architecture
Client requirements	0.5	Section 2.2 – CR-01 to CR-07
CRUD lookup tables	0.5	Section 6 – six lookup tables with full CRUD
Total	5	

1.4 Team Members and Contributions

Each team member owns one of the Phase 1 design patterns together with the related part of the domain model. Each member commits their own work to the team's GitHub repository, so the commit history shows each member's contribution.

Table 2: *Team members and their contributions*

Member	ID	Design pattern	Other responsibilities
Mohamed Magdy Mahmoud	22P0127	Singleton	Database schema, CRUD lookup tables
Mahmoud Nashaat	22P0126	Factory	Donations, recurring donations, campaigns
Mohamed Ghoneem	22P0124	MVC	Views, controllers, Main demo, events, tickets, waiting list
Mohamed Mahdy	22P0089	Strategy	People classes, volunteer certificates
Zakaria Alaa Eisa	22P0128	Observer	Repository setup, client requirements, class diagram, report
Kareem Younis	22P0136	Decorator	Beneficiaries, programs and impact evaluation

1.5 GitHub Repository

The complete source code, class diagram and documentation are available at:
https://github.com/ZakariaEisa/Non-Profit-Organization-Management-System-

2. System Requirements

2.1 Basic Requirements

The basic requirements come from the project brief. Table 2 lists them by module together with the classes that implement each one.

Table 3: *Basic requirements and the classes that implement them*

Module	Requirement	Design elements
Donation Management	Track donor information, donation amounts and donation types	Donor, Donation, DonationType
	Generate donation receipts and tax-related documents	IReceipt, BasicReceipt, TaxInfoReceiptDecorator, QRCodeReceiptDecorator
	Support different donation methods (online, cash/check, in-kind)	DonationFactory, OnlineDonation, CashDonation, InKindDonation
Volunteer Management	Store volunteer information, skills and availability	Volunteer, Skill
	Assign volunteers to tasks by preferences and qualifications	Task, VolunteerAssignment, Volunteer.isQualifiedFor()
	Track volunteer hours and generate certificates	VolunteerAssignment.hoursWorked, VolunteerCertificate
Event Management	Create, manage and track events	Event, EventType, EventController
	Registration, ticketing and attendance tracking	EventRegistration, Ticket, Event.trackAttendance()
	Send event reminders to attendees	IEventObserver, EventReminderObserver
Beneficiary Management	Store beneficiaries and their needs	Beneficiary, BeneficiaryNeed, BeneficiaryCategory
	Track distribution of resources and services	ResourceDistribution, ResourceType
	Monitor and evaluate the impact of programs	Program, ImpactEvaluation
Communication	Email, SMS and social media communication	INotificationStrategy, EmailStrategy, SMSStrategy, SocialMediaStrategy

2.2 Client Requirements

To extend the basic requirements, the team gathered additional requirements from the intended users of the system: donors, volunteers, event coordinators, fundraising staff and administrators. Each client

requirement was given an identifier (CR-01 to CR-07). The identifier is shown in red on the classes that were added or changed for it in the class diagram.

Table 4: *Client requirements collected from users*

ID	Requested by	Requirement	Design elements
CR-01	Donor	Set up a recurring monthly donation instead of donating manually every month.	RecurringDonation (new type created by DonationFactory)
CR-02	Fundraising manager	Run fundraising campaigns with a target amount; progress must update automatically whenever a donation arrives.	Campaign, CampaignProgressObserver
CR-03	Donor	Donate anonymously; the donor name must not appear on public lists or receipts.	Donation.anonymous, getDonorDisplayName()
CR-04	Donors and volunteers	Choose a preferred notification channel (Email, SMS or social media) and be contacted only on it.	NotificationChannel lookup, Person.preferredChannel, NotificationStrategyFactory
CR-05	Event coordinator	When an event is full, new registrations join a waiting list; the first person is promoted automatically when someone cancels.	Event.registerAttendee(), Event.cancelRegistration(), EventRegistration status
CR-06	Volunteer coordinator	Tasks list required skills; only qualified volunteers can be assigned, and preferred event types are considered first.	Task.requiredSkills, Volunteer.preferredEventTypes, isQualifiedFor()
CR-07	Administrator	Manage all lookup values (types, categories, skills, channels) from one admin page without changing code.	LookupEntity, LookupController, AdminView.manageLookups()

3. System Architecture (MVC)

3.1 Model–View–Controller

The system follows the **Model–View–Controller (MVC)** architecture, which separates what the system stores, how it is shown, and how user actions are handled:

- **Model** (`model` package): the domain classes (people, donations, events, beneficiaries, lookups) and the pattern classes that belong to the domain (observers, strategies, receipts). The model has no dependency on views or controllers.
- **View** (`view` package): AdminView, DonorView, VolunteerView and BeneficiaryView. Views only display data and forward user actions to a controller.
- **Controller** (`controller` package): one controller per module. All controllers extend the abstract BaseController, which gives them the shared DatabaseManager instance.

A typical request flows as follows: the donor fills the form in DonorView, DonorView calls DonationController.createDonation(), the controller uses DonationFactory to create the right Donation subclass, attaches the observers and stores it through the model; processing the donation then notifies the observers automatically.

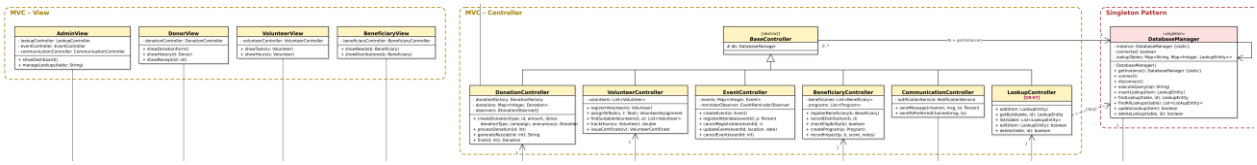


Figure 1: *View and Controller layers with the shared Singleton database manager*

3.2 Package Structure

Table 5: *Packages and their responsibilities*

Package	Layer / pattern	Main classes
model.people	Model	Person (abstract), Donor, Volunteer, Beneficiary
model.donation	Model, Observer subject	Donation (abstract), Cash/Online/InKind/RecurringDonation, Campaign
model.event	Model, Observer subject	Event, EventRegistration, Ticket, Task, VolunteerAssignment
model.certificate	Model	VolunteerCertificate
model.beneficiary	Model	BeneficiaryNeed, ResourceDistribution, Program, ImpactEvaluation
model.lookup	Model, CRUD lookups	LookupEntity (abstract) and 6 lookup tables
model.observer	Observer	IDonationObserver, IEventObserver and 5 concrete observers
model.notification	Strategy	INotificationStrategy, Email/SMS/SocialMediaStrategy, NotificationService
model.receipt	Decorator	IReceipt, BasicReceipt, ReceiptDecorator, QRCode/TaxInfo decorators
factory	Factory	DonationFactory, NotificationStrategyFactory

Package	Layer / pattern	Main classes
database	Singleton	DatabaseManager, schema.sql
controller	Controller	BaseController + 6 controllers
view	View	AdminView, DonorView, VolunteerView, BeneficiaryView

4. Class Diagram

Figure 2 shows the complete Phase 1 class diagram. Packages are drawn as dashed frames, and each design pattern has its own colored frame. The full-resolution diagram is also provided as a separate file (NPO_Class_Diagram_UML.pdf).

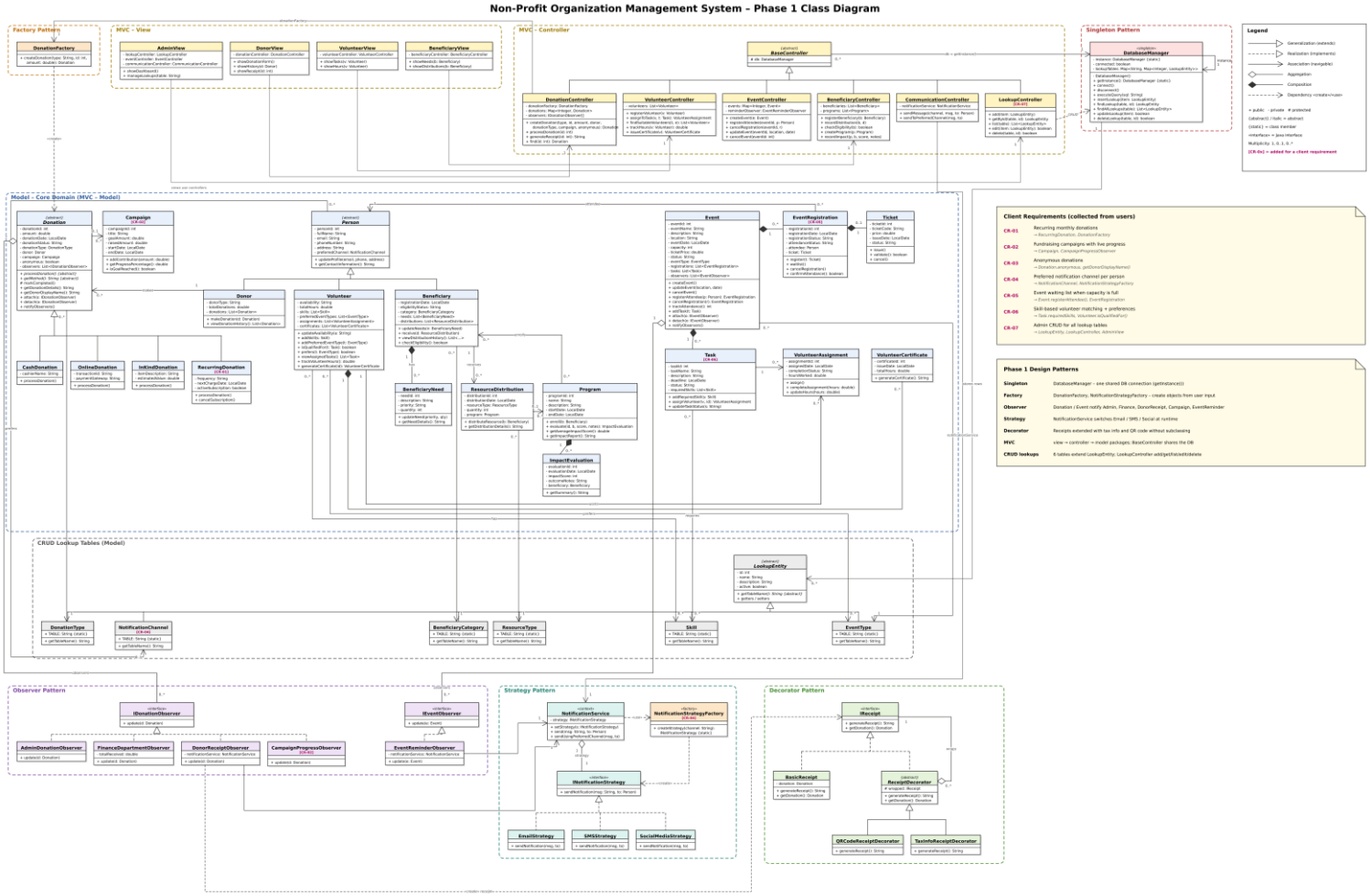


Figure 2: *Complete Phase 1 class diagram of the NPO Management System*

4.1 Core Domain Classes

People

Person is an abstract class that holds the shared contact data (name, email, phone, address) and the preferred notification channel (CR-04). Donor, Volunteer and Beneficiary extend it. A Volunteer keeps skills and preferred event types (CR-06), its task assignments and the certificates it has earned. A Beneficiary belongs to a BeneficiaryCategory, owns its BeneficiaryNeed records and receives ResourceDistribution records. Beneficiaries are enrolled in a Program, whose ImpactEvaluation records measure how much the program helped (a score from 1 to 10).

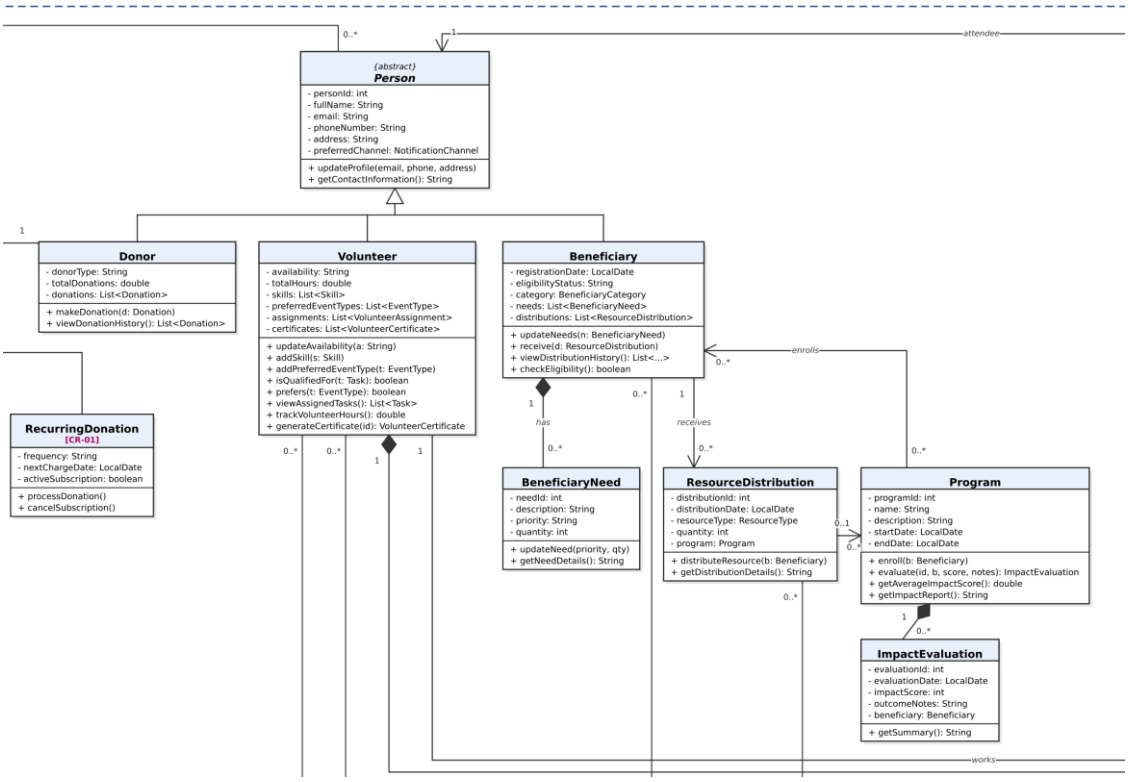


Figure 3: *People and beneficiary classes*

Donations

Donation is abstract and declares `processDonation()`, which every donation method implements in its own way. The four concrete methods are CashDonation, OnlineDonation, InKindDonation and RecurringDonation (CR-01). A donation may support a Campaign (CR-02) and may be anonymous (CR-03). Donation is also the subject of the Observer pattern.

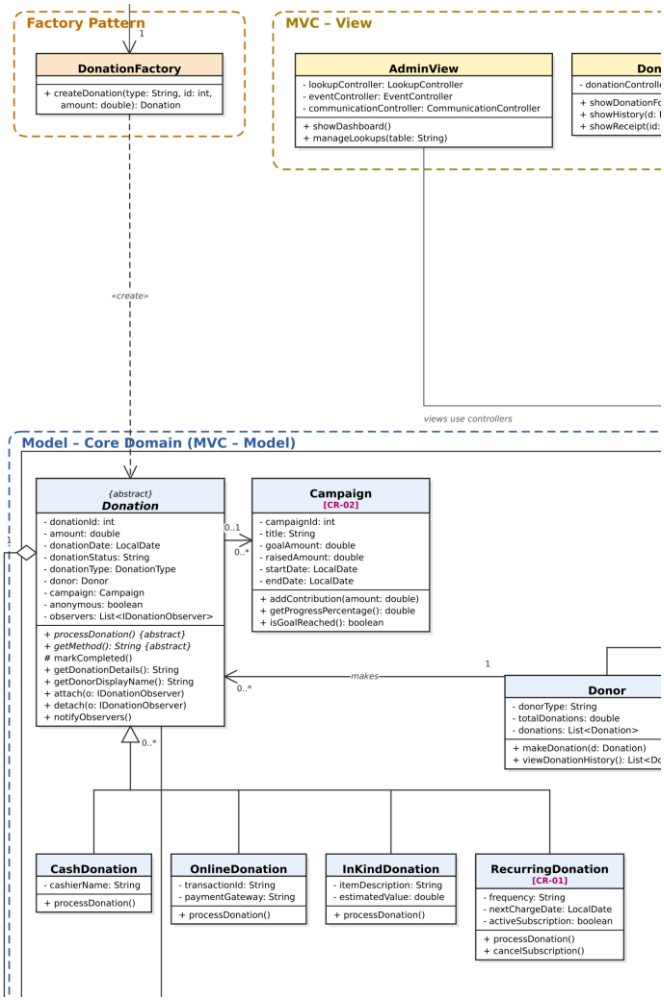


Figure 4: *Donation hierarchy and the DonationFactory*

Events and Volunteering

An Event owns its EventRegistration objects and its Task objects (composition). A confirmed registration receives a Ticket; when capacity is reached the registration is waitlisted instead (CR-05). VolunteerAssignment is the link class between Volunteer and Task and stores the hours worked, which are later used to generate a VolunteerCertificate.

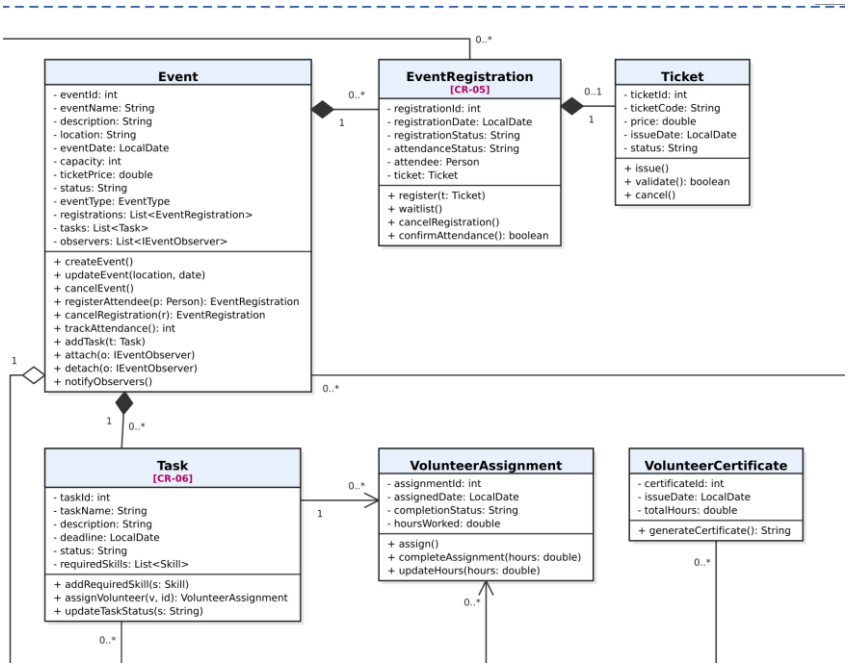


Figure 5: *Event, registration, ticket and volunteer assignment classes*

4.2 Main Relationships

Table 6: *Main relationships in the class diagram*

From	To	Type	Multiplicity	Meaning
Donor, Volunteer, Beneficiary	Person	Generalization	–	Every role is a Person
Donor	Donation	Association	1 → 0..*	A donor makes many donations
Donation	Campaign	Association	0..* → 0..1	A donation can support one campaign
Donation	IDonationObserver	Aggregation	1 → 0..*	Observers attached to a donation
Event	EventRegistration	Composition	1 → 0..*	Registrations live and die with the event
EventRegistration	Ticket	Composition	1 → 0..1	A confirmed registration has a ticket
Event	Task	Composition	1 → 0..*	An event includes tasks
Volunteer / Task	VolunteerAssignment	Association	1 → 0..*	Many-to-many link with hours
Volunteer, Task	Skill	Association	0..* → 0..*	Has / requires skills
Beneficiary	BeneficiaryNeed	Composition	1 → 0..*	Needs belong to one beneficiary
Program	ImpactEvaluation	Composition	1 → 0..*	Evaluations of a program
ReceiptDecorator	IReceipt	Aggregation	0..* → 1	A decorator wraps one receipt
NotificationService	INotificationStrategy	Aggregation	1 → 1	Current sending strategy
BaseController	DatabaseManager	Association	0..* → 1	All controllers share one instance

5. Design Patterns

This section describes each Phase 1 design pattern: the problem it solves in this system, how it is applied, its participants and a short extract of the implementation.

5.1 Singleton Pattern

Aspect	Description
Intent	Ensure a class has only one instance and provide a global point of access to it.
Problem	Every controller needs database access. Creating a new connection per controller wastes resources and can leave the data in inconsistent states.
Solution	DatabaseManager has a private constructor and a static instance created lazily by the synchronized getInstance() method. BaseController obtains the instance once, so all six controllers share it.
Participants	DatabaseManager (singleton); BaseController and its subclasses (clients)

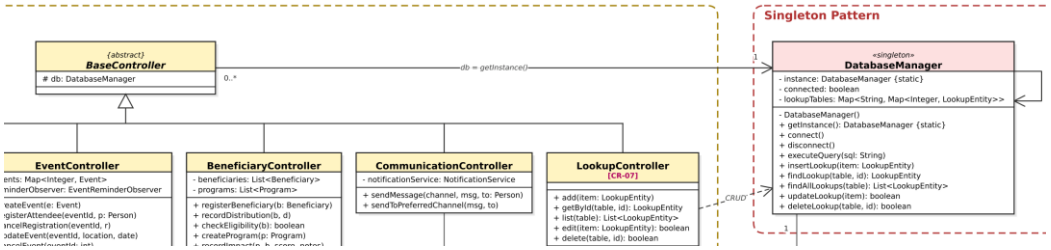


Figure 6: *Singleton DatabaseManager shared by all controllers*

```
public class DatabaseManager {
    private static DatabaseManager instance;

    private DatabaseManager() { }

    public static synchronized DatabaseManager getInstance() {
        if (instance == null) {
            instance = new DatabaseManager();
        }
        return instance;
    }
}

public abstract class BaseController {
    protected final DatabaseManager db = DatabaseManager.getInstance();
}
```

5.2 Factory Pattern

Aspect	Description
Intent	Create objects without exposing the creation logic; the caller asks for a product by name.
Problem	The donation method is chosen by the user at runtime. The controller should not depend on every concrete donation class, and adding a new method should not change controller code.
Solution	DonationFactory.createDonation(type, id, amount) returns the correct Donation subclass. RecurringDonation (CR-01) was added by writing one subclass and one new case. NotificationStrategyFactory applies the same idea to turn a channel name into a strategy (CR-04).

Aspect	Description
Participants	DonationFactory, NotificationStrategyFactory (factories); Donation, INotificationStrategy (products); DonationController, NotificationService (clients)

```
public Donation createDonation(String type, int donationId, double amount) {
    switch (type.trim().toLowerCase()) {
        case "cash": case "check":     return new CashDonation(donationId, amount);
        case "online":                 return new OnlineDonation(donationId, amount);
        case "inkind": case "in-kind": return new InKindDonation(donationId, amount);
        case "recurring":              return new RecurringDonation(donationId, amount);
        default: throw new IllegalArgumentException("Unknown donation type: " + type);
    }
}
```

5.3 Observer Pattern

Aspect	Description
Intent	Define a one-to-many dependency so that when one object changes state, all its dependents are notified automatically.
Problem	When a donation is processed, several parts of the system must react, and when an event changes, its attendees must be informed. Calling each of them directly would tightly couple Donation and Event to every department.
Solution	Donation and Event are subjects with attach(), detach() and notifyObservers(). Observers implement IDonationObserver or IEventObserver and are attached by the controllers. New reactions are added by writing a new observer, without changing the subjects.
Participants	Subjects: Donation, Event. Observers: AdminDonationObserver, FinanceDepartmentObserver, DonorReceiptObserver, CampaignProgressObserver, EventReminderObserver

Table 7: *Observers and their reactions*

Subject	Observer	Reaction
Donation	AdminDonationObserver	Shows the new donation on the admin dashboard
Donation	FinanceDepartmentObserver	Updates the finance department's running total
Donation	DonorReceiptObserver	Builds the receipt (Decorator) and sends it on the donor's preferred channel (Strategy)
Donation	CampaignProgressObserver	Adds the amount to the donation's campaign (CR-02)
Event	EventReminderObserver	Notifies confirmed and waitlisted attendees when the event is updated or cancelled

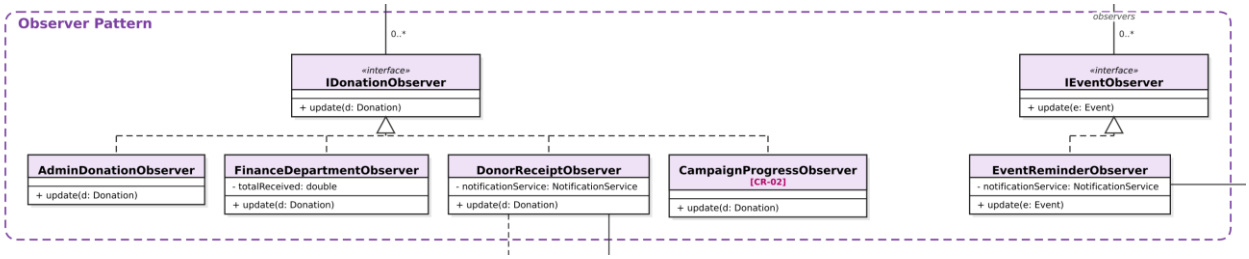


Figure 7: *Observer pattern participants*

```
public void notifyObservers() {            // in Donation and Event
    for (IDonationObserver observer : observers) {
        observer.update(this);
    }
}

protected void markCompleted() {           // called by every processDonation()
    donationStatus = "COMPLETED";
    notifyObservers();
}
```

5.4 Strategy Pattern

Aspect	Description
Intent	Define a family of algorithms, encapsulate each one and make them interchangeable at runtime.
Problem	Messages can be sent by email, SMS or social media, and each person prefers a different channel (CR-04). Using if/else blocks in every sender would duplicate code and make new channels hard to add.
Solution	INotificationStrategy declares sendNotification(). EmailStrategy, SMSStrategy and SocialMediaStrategy implement it. NotificationService (the context) holds the current strategy, which can be replaced with setStrategy(); sendUsingPreferredChannel() selects the strategy from the recipient's preferred channel.
Participants	INotificationStrategy (strategy); EmailStrategy, SMSStrategy, SocialMediaStrategy (concrete strategies); NotificationService (context)

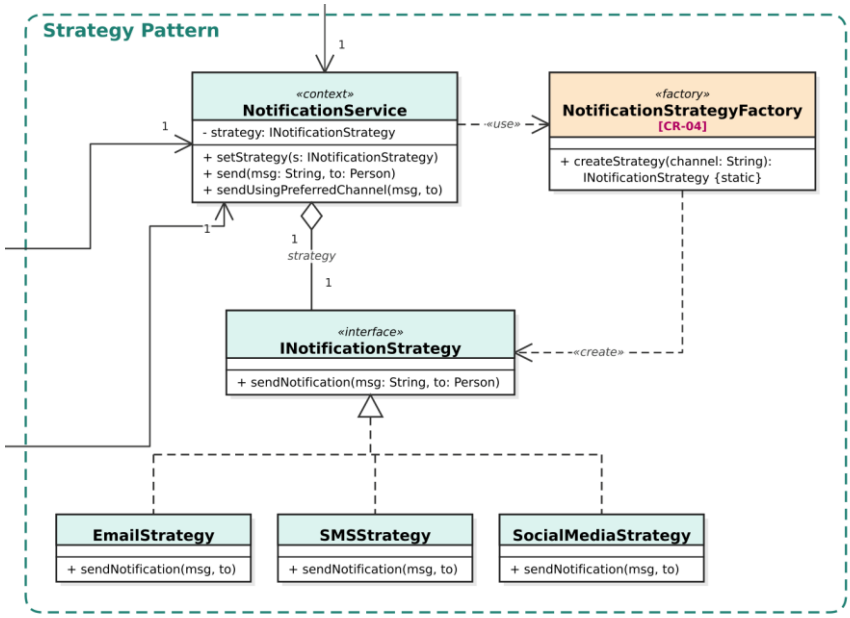


Figure 8: *Strategy pattern for notification channels*

```
public void sendUsingPreferredChannel(String message, Person recipient) {
    String channel = recipient.getPreferredChannel() == null
            ? "Email" : recipient.getPreferredChannel().getName();
    setStrategy(NotificationStrategyFactory.createStrategy(channel));
    send(message, recipient);              // strategy.sendNotification(...)
}
```

5.5 Decorator Pattern

Aspect	Description
Intent	Attach additional responsibilities to an object dynamically, as a flexible alternative to subclassing.
Problem	Receipts need optional additions (tax-deduction information for tax documents, a QR verification code) in any combination. A subclass for every combination would multiply the number of classes.
Solution	IReceipt is the component interface and BasicReceipt the concrete component. The abstract ReceiptDecorator wraps another IReceipt and forwards to it; TaxInfoReceiptDecorator and QRCodeReceiptDecorator each append their own section. Decorators can be stacked in any order.
Participants	IReceipt (component); BasicReceipt (concrete component); ReceiptDecorator (decorator); TaxInfoReceiptDecorator, QRCodeReceiptDecorator (concrete decorators)

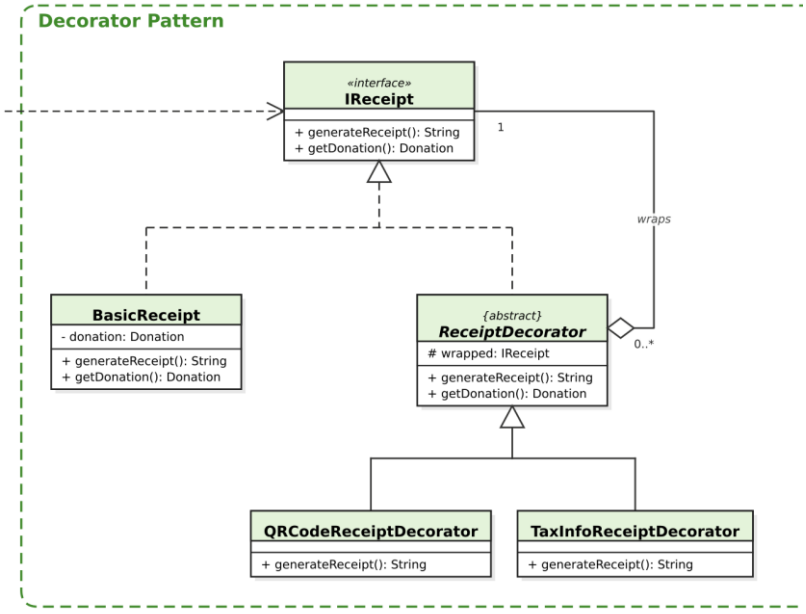


Figure 9: *Decorator pattern for donation receipts*

```
IReceipt receipt = new QRCodeReceiptDecorator(
                       new TaxInfoReceiptDecorator(
                           new BasicReceipt(donation)));
String text = receipt.generateReceipt();
```

Output produced for donation #101:

```
RECEIPT #101
Donor: Sara Ali
Method: Online
Amount: 1500.0 EGP
Date: 2026-11-04
Tax-deductible amount: 1500.0 EGP (keep this receipt for your tax records)
QR verify code: NPO-RCPT-101
```

6. CRUD Lookup Tables

Lookup tables hold small lists of values that administrators must be able to change without touching the code (CR-07). All six lookup classes extend the abstract **LookupEntity** class (id, name, description, active) and each one maps to its own database table.

Table 8: *Lookup tables*

Lookup class	Database table	Example values	Used by
DonationType	donation_type	General, Zakat, Education Fund	Donation
BeneficiaryCategory	beneficiary_category	Orphans, Elderly, Refugees	Beneficiary
EventType	event_type	Fundraiser, Workshop, Outreach	Event, Volunteer (preferences)
Skill	skill	First Aid, Teaching, Driving	Volunteer, Task
ResourceType	resource_type	Food Basket, Medicine, School Kit	ResourceDistribution
NotificationChannel	notification_channel	Email, SMS, Social	Person (preferred channel)

LookupController provides the four CRUD operations for every lookup table, and delegates storage to the Singleton DatabaseManager:

Table 9: *CRUD operations*

Operation	LookupController method	Equivalent SQL
Create	add(item: LookupEntity)	INSERT INTO skill (id, name, description) VALUES (...)
Read	getById(table, id) / list(table)	SELECT * FROM skill WHERE id = ?
Update	edit(item: LookupEntity)	UPDATE skill SET name = ? WHERE id = ?
Delete	delete(table, id)	DELETE FROM skill WHERE id = ?

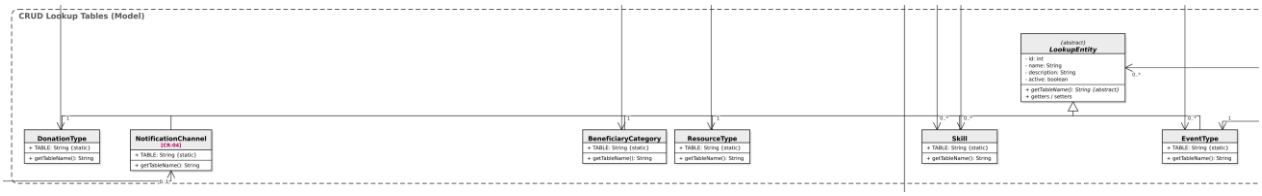


Figure 10: *CRUD lookup tables*

7. Database Design

The relational schema is provided in `src/database/schema.sql`. It contains the six lookup tables and the tables for the domain classes. Many-to-many relationships from the class diagram become link tables.

Table 10: *Main database tables*

Area	Tables
Lookup tables	donation_type, beneficiary_category, event_type, skill, resource_type, notification_channel
People	person, donor, volunteer, beneficiary, volunteer_skill, volunteer_preferred_event_type
Donations	campaign, donation
Events	event, event_registration, ticket, task, task_required_skill, volunteer_assignment, volunteer_certificate
Beneficiaries	beneficiary_need, program, program_enrollment, resource_distribution, impact_evaluation

```
CREATE TABLE skill (
    id          INT PRIMARY KEY,
    name        VARCHAR(100) NOT NULL UNIQUE,
    description VARCHAR(255),
    active      BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE task_required_skill (            -- CR-06
    task_id  INT REFERENCES task(task_id),
    skill_id INT REFERENCES skill(id),
    PRIMARY KEY (task_id, skill_id)
);
```

8. Implementation and Testing

8.1 Implementation

The system is implemented in Java (version 11 or later) with no external libraries. The Phase 1 patterns are fully implemented. `Main.java` walks through every pattern and client requirement step by step. The source code is in the team's GitHub repository (Section 1.5). The project is compiled and run with:

```
mkdir out
javac -d out $(find src -name "*.java")
java -cp out Main
```

8.2 Sample Output

An extract of the demonstration output is shown below.

```
== Singleton - DatabaseManager ==
[DB] Connected.
  same instance everywhere: true
...
- processing donation 101
  Charged online via Paymob (TX-101-80433)
  [Admin dashboard] Donation #101 [Online] 1500.0 EGP by Sara Ali on 2026-11-04 - COMPLETED
  [Finance] recorded 1500.0 EGP, total = 1500.0
  [EMAIL -> sara@mail.com] RECEIPT #101 | Donor: Sara Ali | Method: Online | Amount: 1500.0 EGP |
Date: 2026-11-04 | Tax-deductible amount: 1500.0 EGP (keep this receipt for your tax records)
  [Campaign] 'Back to School' 1500 / 10000 EGP (15.0%)
...
== Events, tickets and waiting list (CR-05) ==
  Sara Ali: CONFIRMED ticket EVT1-T1 (VALID)
  Omar Hassan: CONFIRMED ticket EVT1-T2 (VALID)
  Mona Adel: WAITLISTED
- Omar cancels, the waiting list moves up:
  Omar Hassan: CANCELLED ticket EVT1-T2 (CANCELLED)
  Mona Adel: CONFIRMED ticket EVT1-T3 (VALID)
...
== Volunteer matching, hours and certificate (CR-06) ==
  suitable volunteers: 1 (Mona Adel)
  assign Karim (no first aid): rejected
```

8.3 Test Scenarios

Table 11: *Test scenarios executed with the demonstration program*

#	Scenario	Expected result	Result
1	Call DatabaseManager.getInstance() twice	Both calls return the same object	Passed
2	Add three skills, edit one, delete one	List shows the two remaining skills	Passed
3	Create online, recurring and in-kind donations through the factory	Correct subclass created for each type	Passed
4	Process a donation	Admin, finance, receipt and campaign observers all notified	Passed
5	Two campaign donations (1500 + 300 EGP, goal 10000)	Campaign progress reaches 18%	Passed
6	Anonymous in-kind donation	Shown as “Anonymous donor”	Passed
7	Send the same message by Email, SMS and Social	Strategy switched at runtime for each channel	Passed

#	Scenario	Expected result	Result
8	Register 3 people for an event with capacity 2, then cancel one	Third person waitlisted, then promoted with a new ticket	Passed
9	Event location changed	Attendees notified on their own preferred channel	Passed
10	Assign a volunteer without First Aid to a First Aid task	Assignment rejected; qualified volunteer accepted	Passed
11	Complete a 6-hour assignment and issue a certificate	Certificate shows 6.0 hours	Passed
12	Record a distribution and an impact evaluation (score 8)	Program report shows average impact 8.0/10	Passed

9. Plan for Phase 2

Phase 2 completes the implementation and adds the remaining patterns. The planned use of each pattern is:

Table 12: *Planned Phase 2 patterns*

Pattern	Planned use in the system
Command	Encapsulate admin actions (approve donation, cancel event, assign volunteer) as command objects with undo support
Adapter	Wrap external payment gateways and SMS / social media APIs behind the system's own interfaces
Facade	Provide a single NPOSystemFacade used by the views instead of calling many controllers
Template Method	Define the fixed steps of processing a donation (validate → charge → record → notify) and let each donation type override only its own steps
Iterator	Traverse donations, registrations and waiting lists without exposing their internal collections
State	Model the life cycle of events (Draft, Scheduled, Updated, Cancelled, Completed) and of donations
Proxy	Protect admin operations with an access-control proxy in front of the controllers
Admin pages	Graphical admin dashboard for lookup tables, donations, events and reports

10. Conclusion

Phase 1 delivers a complete object-oriented design of the NPO Management System and a working Java implementation of its core. The class diagram models all five modules from the brief and the seven client requirements collected from users. The Singleton, Factory, Observer, Strategy and Decorator patterns keep the design flexible: new donation methods, communication channels, notification reactions and receipt features can be added without changing existing classes. The MVC architecture separates data, presentation and control, and the six CRUD lookup tables let administrators change reference data without code changes. This design is the foundation for the Phase 2 patterns and the full system implementation.

Deliverables

- This report (`Team13_Phase1_Report.pdf`)
- `NPO_Class_Diagram_UML.pdf` – the complete UML class diagram
- `Source_Code/` – Java source code, README, CLIENT_REQUIREMENTS.md and database schema
- `GitHub_Repo_Link.txt` – link to the team repository
- GitHub repository: https://github.com/ZakariaEisa/Non-Profit-Organization-Management-System-